

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 20%

QUESTIONS:15

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Mathiprakash E 192312085 certify that this submission is my original

work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Math', with a long horizontal stroke extending to the right.

QUESTIONS

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and Data Frame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.
2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.
3. Develop a pseudocode algorithm to combine two or more Data Frames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.
4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.
5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into Data Frames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.

ANSWERS

1. DATASET PREPROCESSING

BEGIN

INPUT: Input_File

OUTPUT: Cleaned_File

IF Input_File does not exist THEN

DISPLAY "File not found"

STOP

END IF

Data $\leftarrow$ READ_FILE(Input_File)

DF $\leftarrow$ CONVERT_TO_DATAFRAME(Data)

DF $\leftarrow$ CREATE_INDEX(DF, Unique_ID)

FOR each column C in DF DO

IF C is numerical THEN

Missing_Values $\leftarrow$ FIND_MISSING(C)

REPLACE Missing_Values with MEDIAN(C)

ELSE IF C is categorical THEN

Missing_Values $\leftarrow$ FIND_MISSING(C)

REPLACE Missing_Values with MODE(C)

END IF

END FOR

DF $\leftarrow$ REMOVE_DUPLICATES(DF, Unique_ID)

FOR each row R in DF DO

IF R contains invalid values THEN

REMOVE R

END IF

END FOR

FOR each numerical column C in DF DO

Min $\leftarrow$ MIN(C)

Max $\leftarrow$ MAX(C)

IF Max $\neq$ Min THEN

$C \leftarrow (C - \text{Min}) / (\text{Max} - \text{Min})$

ELSE

$C \leftarrow 0$

END IF

END FOR

FOR each categorical column C in DF DO

C $\leftarrow$ REMOVE_EXTRA_SPACES(C)

C $\leftarrow$ CONVERT_TO_STANDARD_CASE(C)

```

C ← STANDARDIZE_CATEGORY_VALUES(C)
END FOR

DF ← FILTER_VALID_RECORDS(DF)
DF ← RESET_INDEX(DF)

IF DF contains missing values OR duplicates OR invalid entries THEN
  DISPLAY "Validation failed"
ELSE
  SAVE DF TO Cleaned_File
  DISPLAY "Dataset successfully cleaned"
END IF

END

```

2. STATISTICAL MEASURES

```

BEGIN

INPUT: Dataset
OUTPUT: Mean, Median, Variance, Standard_Deviation

A ← READ_DATASET(Dataset)
A ← CONVERT_TO_NUMERIC_ARRAY(A)

IF SIZE(A) = 0 THEN
  DISPLAY "Empty dataset"
  STOP
END IF

A ← REMOVE_MISSING_VALUES(A)
A ← REMOVE_NON_NUMERIC_VALUES(A)
A ← REMOVE_INFINITE_VALUES(A)

IF SIZE(A) = 0 THEN
  DISPLAY "No valid values available"
  STOP
END IF

A ← FILTER_UNWANTED_VALUES(A)

Mean ← SUM(A) / SIZE(A)

SORT(A)

N ← SIZE(A)

IF N MOD 2 = 1 THEN
  Median ← A[N / 2]
ELSE
  Median ← (A[N / 2 - 1] + A[N / 2]) / 2
END IF

Squared_Differences ← (A - Mean)2

```

```

Variance ← SUM(Squared_Differences) / N
Standard_Deviation ← SQRT(Variance)
Outliers ← IDENTIFY_OUTLIERS(A)

IF SIZE(Outliers) > 0 THEN
  DISPLAY "Outliers detected"
END IF

IF Variance < 0 THEN
  DISPLAY "Invalid variance"
  STOP
END IF

DISPLAY "Mean =", Mean
DISPLAY "Median =", Median
DISPLAY "Variance =", Variance
DISPLAY "Standard Deviation =", Standard_Deviation

END

```

3. COMBINING DATA FRAMES

```

BEGIN

INPUT: DataFrame_A, DataFrame_B
OUTPUT: Merged_DataFrame

CHECK DataFrame_A
CHECK DataFrame_B

IF DataFrame_A is empty OR DataFrame_B is empty THEN
  DISPLAY "Invalid or empty DataFrame"
  STOP
END IF

STANDARDIZE_COLUMN_NAMES(DataFrame_A)
STANDARDIZE_COLUMN_NAMES(DataFrame_B)

DataFrame_A ← REMOVE_EXACT_DUPLICATES(DataFrame_A)
DataFrame_B ← REMOVE_EXACT_DUPLICATES(DataFrame_B)

CHECK Common_Key

IF Common_Key does not exist in both DataFrames THEN
  DISPLAY "Common key not found"
  STOP
END IF

CONVERT DataFrame_A[Common_Key] TO SAME_DATA_TYPE
CONVERT DataFrame_B[Common_Key] TO SAME_DATA_TYPE

```

```

DataFrame_A ← SET_INDEX(DataFrame_A, Common_Key)
DataFrame_B ← SET_INDEX(DataFrame_B, Common_Key)

DataFrame_A ← SORT_BY_INDEX(DataFrame_A)
DataFrame_B ← SORT_BY_INDEX(DataFrame_B)

Merged_DataFrame ← MERGE(DataFrame_A, DataFrame_B, Common_Key, OUTER)

Merged_DataFrame ← REMOVE_DUPLICATES(Merged_DataFrame, Common_Key)

GROUPED_DATA ← GROUP_BY(Merged_DataFrame, Common_Key)

FOR each group G in GROUPED_DATA DO
CHECK for conflicting values
RESOLVE conflicts using predefined rules
END FOR

Merged_DataFrame ← SORT_BY(Merged_DataFrame, Common_Key)

IF Merged_DataFrame contains unexpected missing records THEN
DISPLAY "Completeness check failed"
ELSE IF Merged_DataFrame contains duplicate keys THEN
DISPLAY "Duplicate check failed"
ELSE
DISPLAY "Merge successful"
END IF

VERIFY record counts
VERIFY common key integrity
VERIFY column consistency

RETURN Merged_DataFrame

END

```

4. FILTERING AND SORTING DATA

```

BEGIN

INPUT: Dataset
OUTPUT: Filtered_and_Sorted_Data

DF ← READ_DATASET(Dataset)

IF DF is empty THEN
DISPLAY "Dataset is empty"
STOP
END IF

DF ← CREATE_INDEX(DF, Record_ID)

Condition_1 ← DF[Value] >= Minimum_Value
Condition_2 ← DF[Value] <= Maximum_Value

```

```

Condition_3 ← DF[Category] IN Valid_Categories
Condition_4 ← DF[Status] = Valid_Status

Combined_Condition ← Condition_1 AND Condition_2 AND Condition_3 AND Condition_4

Filtered_Data ← DF[Combined_Condition]

IF SIZE(Filtered_Data) = 0 THEN
  DISPLAY "No records satisfy the conditions"
  STOP
END IF

Filtered_Data ← SORT(Filtered_Data, Sort_Column, ASCENDING)

Filtered_Data ← RESET_INDEX(Filtered_Data)

VERIFY every record satisfies Combined_Condition

IF SIZE(Filtered_Data) is large THEN
  PROCESS DATA IN CHUNKS
  APPLY FILTERING TO EACH CHUNK
  COMBINE FILTERED CHUNKS
  SORT FINAL RESULT
END IF

RETURN Filtered_Data

END

```

5. COMPLETE DATA ANALYSIS WORKFLOW

```

BEGIN

INPUT: Input_File
OUTPUT: Summary_Table, Plot

IF Input_File does not exist THEN
  DISPLAY "Input file not found"
  STOP
END IF

TRY
  Data ← READ_FILE(Input_File)
CATCH File_Error
  DISPLAY "Error while reading file"
  STOP
END TRY

IF Data is empty THEN
  DISPLAY "No data available"
  STOP
END IF

DF ← CONVERT_TO_DATAFRAME(Data)

```

```

CHECK column names
CHECK data types
CHECK missing values
CHECK duplicate records

DF ← REMOVE_DUPLICATES(DF)

FOR each required column C DO
  CONVERT C TO APPROPRIATE_DATA_TYPE
END FOR

DF ← HANDLE_MISSING_VALUES(DF)

DF ← FILTER_INVALID_RECORDS(DF)

DF ← SORT_BY(DF, Required_Column)

GROUPED_DATA ← GROUP_BY(DF, Grouping_Column)

Summary_Table ← AGGREGATE(
  GROUPED_DATA,
  COUNT,
  SUM,
  MEAN,
  MIN,
  MAX
)

Summary_Table ← SORT(Summary_Table, Required_Metric)

IF Summary_Table is empty THEN
  DISPLAY "No summarized data available"
  STOP
END IF

SELECT required columns for visualization

CREATE_PLOT(
  Summary_Table,
  X_AXIS,
  Y_AXIS,
  Plot_Type
)ADD title
ADD axis labels
ADD legend if requiredDISPLAY PlotTRY
SAVE Summary_Table TO Summary_File
SAVE Plot TO Plot_File
CATCH File_Error
DISPLAY "Error while saving output"
END TRY

DISPLAY "Data analysis completed successfully"

RETURN Summary_Table, Plot

END

```